VB The Language

Visual Basic 6.0 (1998) - Study Notes for OAVS Computer Teacher Examination

Contents: 1 Variables, Constants, Datatypes, UDT | 2 Scope | 3 Control Flow | 4 Looping | 5 Arrays | 6 Procedures & Functions | 7 VB Functions (Date, Mathematical, String, Conversion)

1. Variable, Constants, Datatypes, User Defined Datatypes

1.1 Variable

A **variable** is a named location in memory that holds a value which **can change** while the program runs. The name is used to store and retrieve the value. **Purpose:** to keep input, intermediate results and output.

Syntax: `Dim variableName [As DataType]` (declaration), then `variableName = value` (initialization / assignment).

Example - declaring and initialising variables

```
Dim age As Integer            'declaration
age = 18                      'initialization (assignment)
Dim city As String, fee As Currency   'several variables: EACH needs its own As clause
Dim n As Integer: n = 5       'colon joins statements on one line
```

Rules for variable names:

- Must begin with a letter; later characters may be letters, digits or underscore (_).
- Maximum 255 characters; no spaces, dots or special characters (`- + * @ #` etc.).
- Cannot be a reserved word (`Print, Dim, End, Name` ...); must be unique within its scope.
- Not case-sensitive: `Total` and `TOTAL` are the same variable.

Name	Valid?	Reason
`roll_no`	Valid	Letters and underscore
`Marks1`	Valid	Digit allowed after the first letter
`2ndYear`	Invalid	Starts with a digit
`first name`	Invalid	Contains a space
`my-var`	Invalid	Hyphen is the minus operator
`Print`	Invalid	Reserved word

Default (initial) values

A variable that has been declared but not yet assigned holds a default value that depends on its data type:

Data type	Default value
Numeric (Byte, Integer, Long, Single, Double, Currency)	**0**
String	**""** (empty string)
Boolean	**False**
Date	**0** (shown as 12:00:00 AM)
Variant	**Empty**
Object	**Nothing**

Implicit vs explicit declaration

- **Implicit declaration:** if a name is used without `Dim`, VB silently creates it as a **Variant**.
- **Problem:** a typing mistake in a variable name creates a new, unwanted variable instead of showing an error.
- **Explicit declaration:** write `Option Explicit` at the top of the module, or tick *Require Variable Declaration* in *Tools* > *Options*. VB then reports an error for every undeclared variable.

Type-declaration character	Data type	Example
%	Integer	`Dim a%`
&	Long	`Dim b&`
!	Single	`Dim c!`
#	Double	`Dim d#`
@	Currency	`Dim e@`
$	String	`Dim f$`

COMMON MISTAKE: `Dim a, b As Integer` makes only **b** an Integer; **a** becomes a Variant. Write `Dim a As Integer, b As Integer`.

1.2 Constants

A **constant** is a named value that **cannot change** while the program runs. **Purpose:** readable code, one place to change a fixed value (define it once as a constant; changing that one line updates it everywhere), and protection from accidental modification.

Syntax: `[Public|Private] Const name [As Type] = expression`

Syntax and example - declaring constants

```
Const PI As Double = 3.14159
Const COMPANY = "OAVS"            'type inferred from the value
Const TAX = 0.18, MAXSTUDENTS = 40
PI = 3.14                         'ERROR: Assignment to constant not permitted
```

- The value must be a literal or an expression of other constants - a function result or variable is not allowed.
- Difference from a variable: a variable's value can be reassigned; a constant's value is fixed at design time.
- Constants are of **two kinds**: user-defined and intrinsic (built-in), explained below.

Kind 1: User-defined constants

- **Created by the programmer** with the `Const` statement (as shown above), with a name and value of their choice.
- Scope follows the declaration place: inside a procedure = local to it; `Private Const` in Declarations = that module only; `Public Const` in a standard module = whole project.
- Examples: `PI`, `TAX`, `MAXSTUDENTS` - values specific to your own program.

Kind 2: Intrinsic (built-in) constants

- **Already defined by Visual Basic** - no `Const` declaration is needed; just use the name. Their value cannot be changed.
- Most start with the prefix **vb** (e.g. `vbYes`), so they are easy to recognise. Each stands for a fixed number or text.
- **Why use them:** the name is readable and meaningful, while the raw number (6, 36, 255 ...) is hard to remember.

Group	Constant	Value	Meaning / use
Logical	`True / False`	-1 / 0	Condition is true / false
Text	`vbCrLf`	Chr(13) + Chr(10)	New line inside text (carriage return + line feed)
	`vbTab`	Chr(9)	Tab space inside text
MsgBox buttons	`vbOKOnly`	0	Shows only the OK button
	`vbOKCancel / vbYesNo`	1 / 4	OK + Cancel buttons / Yes + No buttons
MsgBox icons	`vbCritical`	16	Red error icon
	`vbQuestion`	32	Question mark icon
	`vbExclamation / vbInformation`	48 / 64	Warning icon / Information icon
MsgBox result	`vbOK / vbCancel`	1 / 2	Button the user clicked (returned by MsgBox)
	`vbYes / vbNo`	6 / 7	Button the user clicked (returned by MsgBox)
Colours	`vbRed / vbGreen / vbBlue`	255 / 65280 / 16711680	Red / Green / Blue colour

Program 1 - intrinsic constants

```
Dim ans As Integer
MsgBox "Line 1" & vbCrLf & "Line 2"                    'text on two lines
ans = MsgBox("Save the file?", vbYesNo + vbQuestion)    'same as MsgBox(..., 36)
If ans = vbYes Then
    Print "You clicked Yes"
Else
    Print "You clicked No"
End If
Text1.BackColor = vbRed                                 'colour constant
```

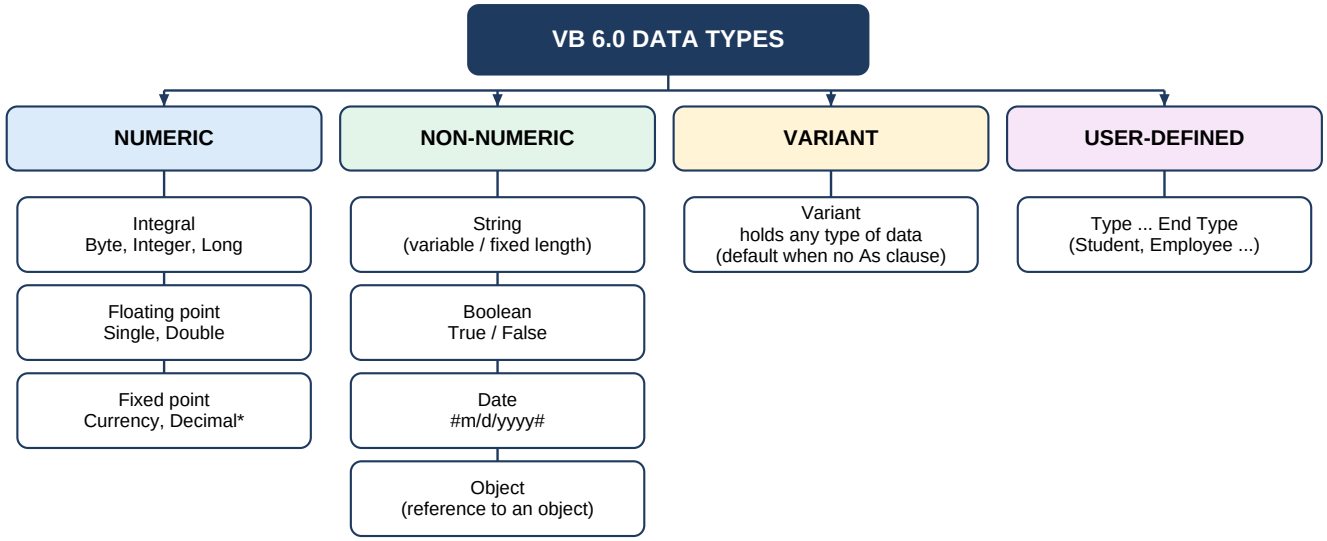
```
OUTPUT
Box 1: Line 1 and Line 2 on separate lines
Box 2: Save the file?  [Yes] [No]  (question icon)
Yes clicked -> You clicked Yes
```

User-defined constant	Intrinsic constant
Declared by the programmer using `Const`	Already provided by VB; no declaration
Name and value chosen by you	Name and value fixed by VB (usually starts with `vb`)
Meaning specific to your program (`TAX`)	General use (`vbYes`, `vbCrLf`)

User-defined constant	Intrinsic constant
Can be changed by editing the Const line	Cannot be changed at all

1.3 Data Types

A **data type** defines what kind of value a variable can hold, how much memory it uses and which operations are valid. Choosing the smallest suitable type saves memory and avoids errors.



** Decimal can only be used through a Variant (CDec). Boolean/Date/Object are not numeric types.*

Fig 1: Classification of VB 6.0 data types

Type	Size	Range / holds	Typical use
Byte	1 byte	0 to 255	Small whole numbers, binary data
Boolean	2 bytes	True (-1) or False (0)	Flags, conditions
Integer	2 bytes	-32,768 to 32,767	Counters, loop variables, ages
Long	4 bytes	-2,147,483,648 to 2,147,483,647	Large whole numbers (e.g. factorial)
Single	4 bytes	about 7 significant digits (decimals)	Marks, averages, prices
Double	8 bytes	about 15 significant digits	Scientific / precise calculations
Currency	8 bytes	4 fixed decimal places, up to 15 digits before the point	Money (no rounding error)
Date	8 bytes	1 Jan 100 to 31 Dec 9999 (with time)	Dates and times, written #8/15/2026#
String	variable (fixed: set by * n)	Text; `String * 10` = fixed length 10, space-padded	Names, addresses
Object	4 bytes	Reference to an object (form, control)	Working with objects
Variant	16 bytes + (22 + string length for text)	Any type; default if no As clause	Unknown / mixed data; slower, uses more memory

Valid and invalid assignments

Statement	Result	Explanation
`Dim n As Integer: n = 3.7`	Valid, n = 4	Decimal part is rounded (to nearest; .5 rounds to even)
`n = "25"`	Valid, n = 25	Numeric text converted automatically
`n = 40000`	Invalid	Run-time error 6: Overflow (Integer max 32767)
`n = "abc"`	Invalid	Run-time error 13: Type mismatch
`Dim b As Boolean: b = 5`	Valid, b = True	Any non-zero number is True; 0 is False
`Dim d As Date: d = #8/15/2026#`	Valid	Date literals are enclosed in # #

Program 2 - variables and a constant

```
Const PI As Double = 3.14159           'General Declarations
Private Sub Form_Click()
    Dim r As Integer, area As Double
    r = 5
    area = PI * r * r
    Print "Radius = " & r
    Print "Area = " & area
    Print "Circumference = " & 2 * PI * r
End Sub
```

```
OUTPUT
Radius = 5
Area = 78.53975
Circumference = 31.4159
```

1.4 User Defined Data Types (Type ... End Type)

A **user-defined type (UDT)** groups several related items of **different** data types into one record-like unit (e.g. roll number, name and marks of a student). Its items are called **members** (fields). **Purpose:** to treat a real-world entity as a single variable.

Syntax of a user-defined type

```
[Private|Public] Type TypeName
    member1 As DataType
    member2 As DataType
End Type                        'declare the type once, then:
Dim v As TypeName               'create variables of that type
v.member1 = value               'access a member with the dot operator
```

- A `Type` is declared only in the **Declarations** section, never inside a procedure.
- In a standard module it may be `Public` (default); in a Form or Class module it **must be** `Private`.
- Members cannot be given initial values; use fixed-length strings (`String * 15`) for text members.
- Assigning one variable of the type to another (`s2 = s1`) copies **all** members; `With ... End With` avoids repeating the variable name. Arrays of a UDT are allowed (`Dim cls(1 To 30) As Student`).

Program 3 - user-defined type (Student record)

```
Private Type Student                    'General Declarations of the form
    RollNo   As Integer
    StuName  As String * 15
    Marks    As Single
End Type

Private Sub Form_Click()
    Dim s1 As Student, s2 As Student
    With s1
        .RollNo = 101
        .StuName = "Amit"
        .Marks = 87.5
    End With
    s2 = s1                             'copy all members
    s2.Marks = 90                       'changes only s2
    Print "s1: " & s1.RollNo & " " & Trim(s1.StuName) & " " & s1.Marks
    Print "s2: " & s2.RollNo & " " & Trim(s2.StuName) & " " & s2.Marks
End Sub
```

```
OUTPUT
s1: 101 Amit 87.5
s2: 101 Amit 90
```

Invalid use	Why it fails
`s1 = "Amit"`	A UDT variable cannot take a single value; assign to a member (`s1.StuName`)
`s1.Grade = "A"`	Member not declared: "Method or data member not found"
`Type declared inside a Sub`	Compile error: Type must be in the Declarations section
`Public Type in a Form`	Compile error: must be `Private` in a form/class module

2. Scope of Variable (Global, Local, Modular, Static)

Scope = *where* in the program a variable can be seen and used (visibility / accessibility). **Lifetime** = *how long* the variable exists in memory and keeps its value. A variable's scope is decided by **where** and **with which keyword** it is declared.

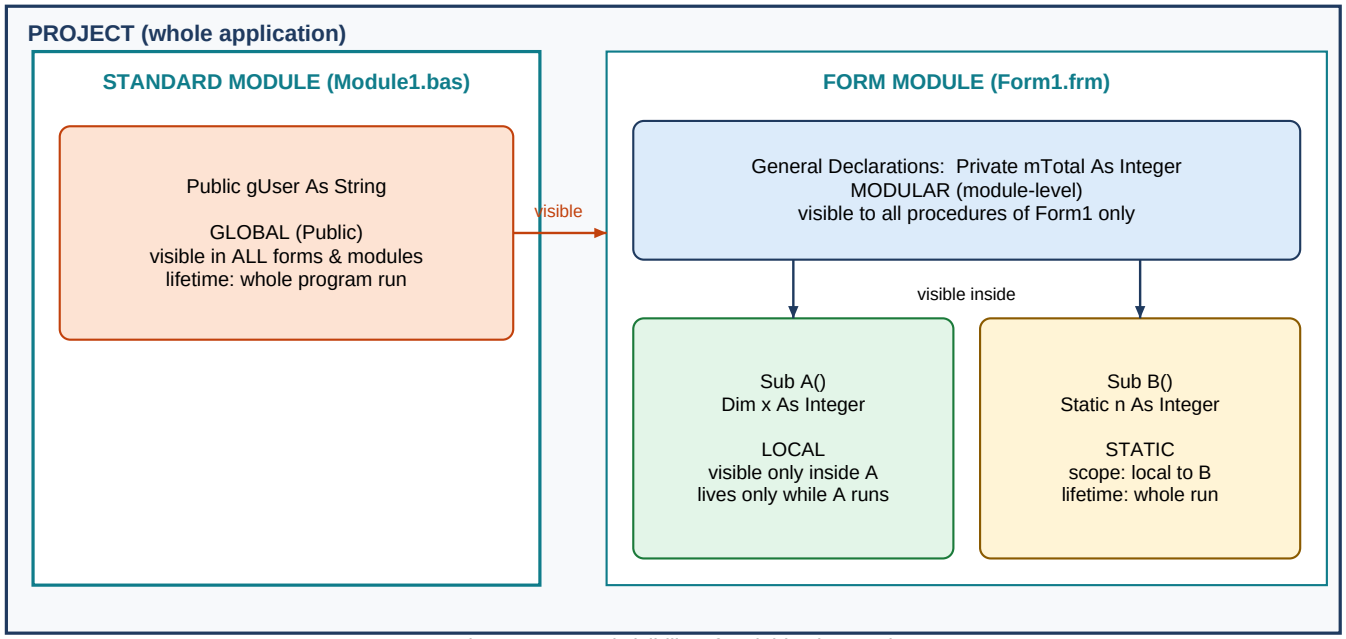


Fig 2: Scope and visibility of variables in a project

2.1 Local variable

- **Declared:** with `Dim` **inside** a procedure.
- **Scope / visibility:** only that procedure. **Lifetime:** created when the procedure starts, destroyed when it ends; every call starts again with the default value (0, "" ...).
- **Use:** temporary values, counters, loop variables.

Program 4 - local variable

```
Private Sub Command1_Click()
    Dim count As Integer          'local
    count = count + 1
    MsgBox "Count = " & count
End Sub
```

```
OUTPUT
Every click shows:  Count = 1      (count is re-created as 0 on each call)
```

2.2 Modular (module-level) variable

- **Declared:** with `Dim` or `Private` in the **General Declarations** section of a form or module.
- **Scope:** every procedure of **that** form/module, but not other forms. **Lifetime:** as long as the form/module is loaded.
- **Use:** data shared between procedures of one form (running total, flag).

Program 5 - modular variable

```
Private total As Integer              'General Declarations - modular

Private Sub cmdAdd_Click()
    total = total + 10
End Sub
Private Sub cmdShow_Click()
    MsgBox "Total = " & total
End Sub
```

```
OUTPUT
Click Add three times, then Show:   Total = 30      (value is kept between calls)
```

2.3 Global (Public) variable

- **Declared:** with `Public` (older keyword `Global`) in the Declarations of a **standard module (.bas)**.
- **Scope:** the whole project - every form and module. **Lifetime:** the entire run of the application.

- **Use:** data needed everywhere (logged-in user, connection settings). Avoid overuse: any code can change it.
- A `Public` variable declared in a *Form* becomes a property of that form and is accessed as `Form1.varName`.

Program 6 - global variable

```
'--- Module1.bas ---
Public gUser As String

'--- Form1 ---                          '--- Form2 ---
gUser = "Admin"                         MsgBox "Welcome " & gUser
```

```
OUTPUT
Welcome Admin        (Form2 reads the value set in Form1)
```

2.4 Static variable

- **Declared:** with `Static` **inside** a procedure (`Static Sub` makes all its variables static).
- **Scope:** only that procedure (like a local). **Lifetime:** the whole program run - the value is **retained** between calls.
- **Use:** counters or flags that must remember their value but need not be seen elsewhere.

Program 7 - static variable

```
Private Sub Command1_Click()
    Static count As Integer       'static
    count = count + 1
    MsgBox "Count = " & count
End Sub
```

```
OUTPUT
Click 1: Count = 1      Click 2: Count = 2      Click 3: Count = 3
```

Name conflict: if a local variable and a modular/global variable have the **same name**, the procedure uses its **own local** variable. The other variable is hidden there and is not changed. Example: if `x` is declared at module level and again with `Dim x` inside a procedure, then `x = 5` in that procedure changes only the local x.

2.5 Comparison and Scope vs Lifetime

	Local	Modular	Global (Public)	Static
Keyword	`Dim`	`Dim / Private`	`Public / Global`	`Static`
Declared in	Inside a procedure	General Declarations of a form/module	General Declarations of a .bas module	Inside a procedure
Visible in	That procedure only	All procedures of that module	Entire project	That procedure only
Lifetime	While the procedure runs	While the module/form is loaded	Whole program run	Whole program run
Value kept between calls?	No	Yes	Yes	Yes
Typical use	Temporary work	Shared within a form	Application-wide data	Call counters

Scope	Lifetime
Where the variable can be accessed (space / code region)	**How long** the variable exists and keeps its value (time)
Decided by place and keyword of declaration	Decided by when it is created and destroyed
Static variable: scope is **narrow** (one procedure)	Static variable: lifetime is **long** (whole run) - proves the two are independent

3. Control Flow Statements (IF, IIF, Select Case)

Control flow statements let a program **decide** which statements to run, based on a condition. A condition is an expression that is **True** or **False**, built with relational operators (`= <> < > <= >=`) and logical operators (`And, Or, Not`). Note: VB 6.0 evaluates **both** sides of `And`/`Or` (no short-circuit).

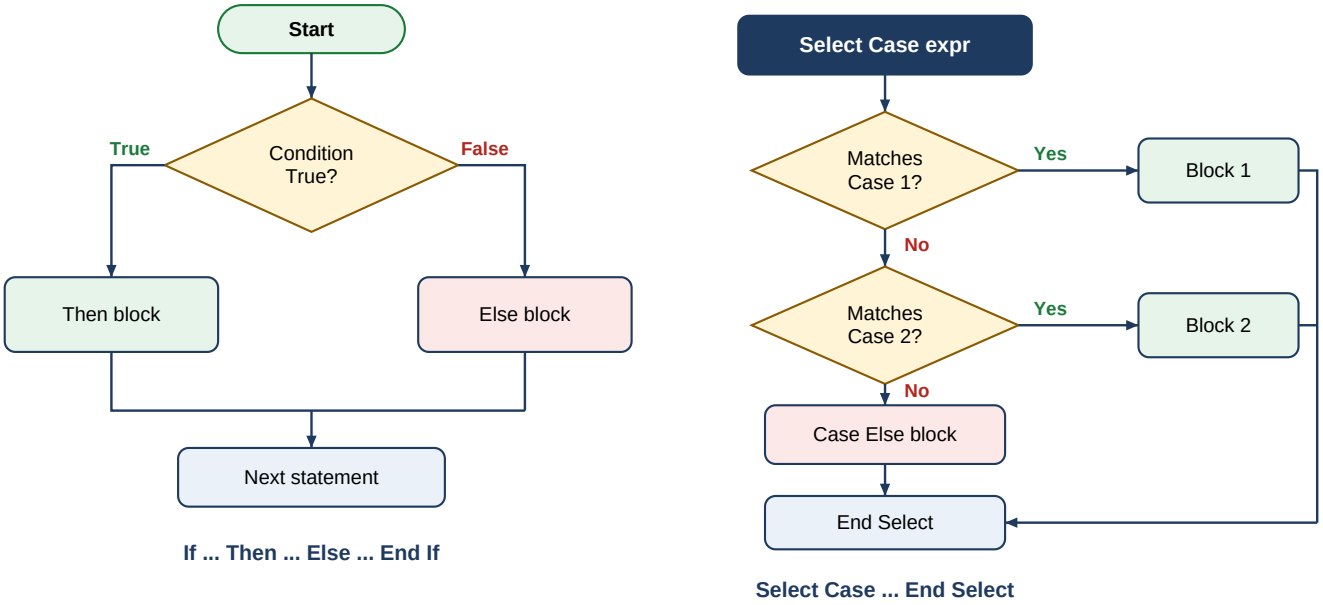


Fig 3: Decision-making flowcharts - If...Then...Else and Select Case

3.1 If Statement (If-Then, If-Then-Else, ElseIf)

Syntax - If statement

```
If condition Then statement                    'single-line (no End If)

If condition Then                              'block form
    statements
[ElseIf condition2 Then
    statements]
[Else
    statements]
End If
```

Execution: conditions are tested from top to bottom; the **first True** block runs, the rest are skipped; `Else` runs only if all conditions are False. Use for one or two-way and multi-way decisions.

Program 8 - If...ElseIf (grade)

```
Dim marks As Integer, grade As String
marks = 72
If marks >= 90 Then
    grade = "A"
ElseIf marks >= 75 Then
    grade = "B"
ElseIf marks >= 60 Then
    grade = "C"
ElseIf marks >= 40 Then
    grade = "D"
Else
    grade = "F"
End If
Print "Marks " & marks & " -> Grade " & grade
```

```
OUTPUT
Marks 72 -> Grade C
```

3.2 Nested If

An **If inside another If**. The inner If is tested only when the outer condition allows it. Each block needs its own `End If`.

Program 9 - nested If (largest of three)

```
Dim a As Integer, b As Integer, c As Integer, big As Integer
a = 15: b = 42: c = 27
If a > b Then
    If a > c Then
        big = a
    Else
        big = c
    End If
Else
    If b > c Then
        big = b
    Else
        big = c
    End If
End If
Print "Largest = " & big
```

```
OUTPUT
Largest = 42
```

3.3 IIf() function

Syntax: `IIf(condition, TruePart, FalsePart)` - a **function** that returns one of two values. It is a compact way to choose a value inside an expression.

Program 10 - IIf()

```
Dim marks As Integer
marks = 35
Print IIf(marks >= 40, "Pass", "Fail")
```

```
OUTPUT
Fail
```

Point	If ... Then ... Else (statement)	IIf() (function)
Nature	Statement that **executes** blocks of code	Function that **returns a value**
Evaluation	Only the chosen branch is executed	**Both** TruePart and FalsePart are always evaluated
Can contain	Many statements, ElseIf, nesting	Only two expressions, no statements
Use inside expression	No	Yes, e.g. `Print IIf(x > 0, "+", "-")`
Risk	None	`IIf(d <> 0, 10 / d, 0)` with d = 0 still evaluates 10/d: **Error 11, Division by zero**

3.4 Select Case

Selects **one** block out of many by comparing a **single expression** (evaluated once) with a list of `Case` values. Cleaner than a long ElseIf ladder when one expression is tested against many values or ranges.

Syntax - Select Case

```
Select Case expression
    Case value1                  'single value
    Case v2, v3, v4              'list of values
    Case 10 To 20                'range
    Case Is > 50                 'comparison
    Case Else                    'optional: nothing matched
End Select
```

Program 11 - Select Case (season)

```
Dim m As Integer
m = 4
Select Case m
    Case 12, 1, 2
        Print "Month " & m & " -> Winter"
    Case 3 To 5
        Print "Month " & m & " -> Summer"
    Case 6 To 9
        Print "Month " & m & " -> Monsoon"
    Case 10, 11
        Print "Month " & m & " -> Autumn"
    Case Else
        Print "Invalid month"
End Select
```

```
OUTPUT
Month 4 -> Summer
```

- Only the **first matching** Case runs; there is no fall-through, so no `break` is needed.
- Works with numbers and strings (`Case "A" To "F"`); use `Case Else` to catch unexpected values.

3.5 Common mistakes

Mistake	Correct form / effect
Forgetting `Then` or `End If` in block form	Compile error "Block If without End If"; single-line If needs no End If
Writing `Else If` (two words) as if it were ElseIf	Starts a **new nested** If that needs its own End If
Using IIf with an expression that can fail	Both parts run: use If...Then...Else instead
Overlapping Case ranges, e.g. `Case 1 To 10` then `Case 5`	Value 5 matches the first Case only; the second never runs
Wrong order in an ElseIf ladder (`>= 40` tested before `>= 90`)	Higher grades never reached; test the most specific condition first
No `Case Else` for unexpected input	Invalid values are silently ignored

4. Looping, Nesting (Do While, Do Until, For, For Each, Exit)

4.1 Loop - Meaning and Types

Definition: A **loop** (iteration) is a control structure that **repeats a block of statements** as long as, or until, a condition is satisfied. It saves writing the same code many times and lets a program process lists of data or repeat until the user gives valid input.

Parts of a loop

Part	Meaning	Example
Initialization	Gives the control variable its starting value	`i = 1`
Condition	Decides whether another pass is made	`i <= 5`
Body	The statements that are repeated	`Print i`
Update	Changes the control variable so the loop can finish	`i = i + 1`

Types of loops in VB 6.0

Group	Loop statements	Used when
Condition-based (Do loops)	`Do While...Loop`, `Do...Loop While`, `Do Until...Loop`, `Do...Loop Until`	Number of repetitions is **not known** in advance
Count-based (For loops)	`For...Next`, `For Each...Next`	Number of repetitions is **known**, or every item of a group is processed
Supporting features	Nested loops, `Exit For`, `Exit Do`, `Exit Sub`	Loop inside a loop; leaving a loop or procedure early

Pre-test and post-test loops

- **Pre-test:** the condition is checked **before** the body (Do While, Do Until, For). The body may run **zero times**.
- **Post-test:** the condition is checked **after** the body (Do...Loop While / Until). The body always runs **at least once**.

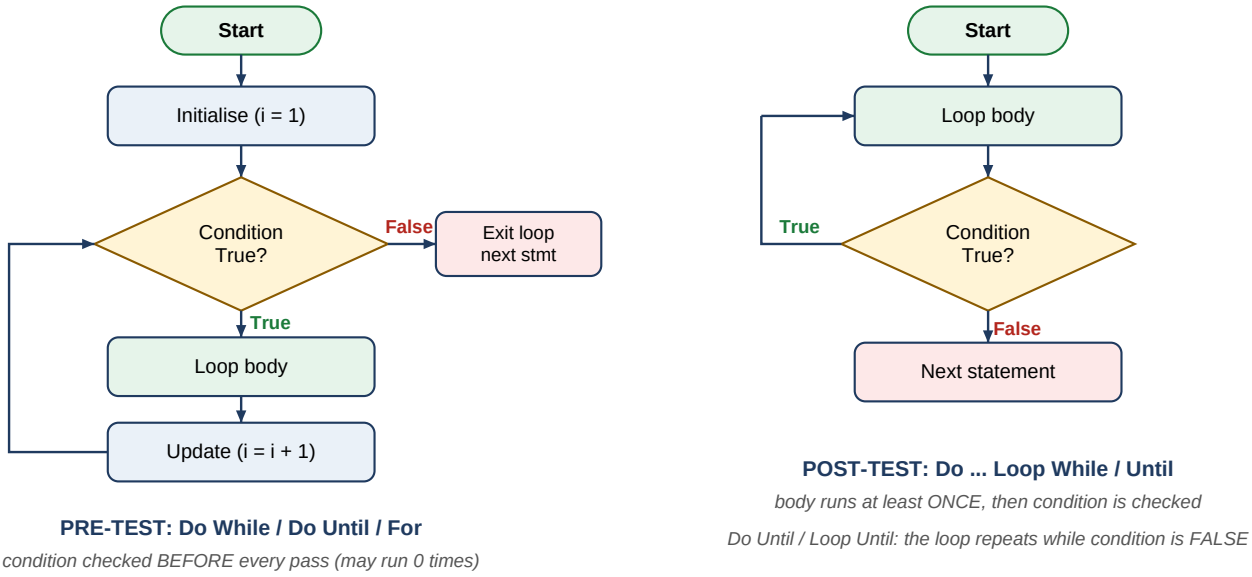


Fig 4: Execution flow of pre-test and post-test loops

4.2 Do While ... Loop

Definition: Repeats the body **while the condition is True**. The condition is tested **before** every pass (pre-test), so if it is False at the start the body never runs.

Syntax

```
Do While condition
    statements
Loop
```

How it works

1. The condition is tested.
2. If **True**: the body runs, then control returns to step 1.
3. If **False**: the loop ends and execution continues after `Loop`.

Program 12 - Do While - print 1 to 5

```
Dim i As Integer, s As String
i = 1                           'initialization
Do While i <= 5                 'condition
    s = s & i & " "             'body
    i = i + 1                   'update
Loop
Print s
```

```
OUTPUT
1 2 3 4 5
```

4.3 Do ... Loop While

Definition: Runs the body **first**, then tests the condition and repeats **while it is True** (post-test). The body therefore executes **at least once**, even if the condition is False from the start.

Syntax

```
Do
    statements
Loop While condition
```

How it works

1. The body runs once.
2. The condition is tested.
3. If **True**: go back to step 1. If **False**: the loop ends.

Program 13 - Do ... Loop While - body runs once

```
Dim i As Integer
i = 10
Do
    Print "i = " & i          'runs once although i <= 5 is False
    i = i + 1
Loop While i <= 5
```

```
OUTPUT
i = 10
```

TIP: Use a post-test loop when the body must run at least once, e.g. asking for input before checking it.

4.4 Do Until ... Loop and Do ... Loop Until

Definition: Repeats the body **while the condition is False** and stops when it becomes **True** - the opposite of Do While. **`Do Until...Loop`** tests before the body; **`Do...Loop Until`** tests after it (runs at least once).

Syntax

```
Do Until condition              'pre-test
    statements
Loop

Do                              'post-test
    statements
Loop Until condition
```

How it works

1. The condition is tested (before or after the body, depending on the form).
2. If **False**: the body runs and the test is repeated.
3. If **True**: the loop ends.

Program 14 - Do Until - doubling until above 50

```
Dim n As Integer
n = 1
Do Until n > 50
    n = n * 2                   'n = 2, 4, 8, 16, 32, 64
Loop
Print "n = " & n
```

```
OUTPUT
n = 64
```

4.5 For ... Next

Definition: Repeats the body a **fixed number of times** using a counter that is initialised, tested and updated automatically.

Syntax

```
For counter = start To end [Step increment]
    statements
Next [counter]
```

Parts and rules

- `start` is the first value and `end` the last value of the counter; `Step` is added after each pass (default **1**).
- A **negative Step** counts down, so start must be larger than end (`For i = 5 To 1 Step -1`).
- Number of passes = Int((end - start) / step) + 1. After the loop finishes normally, the counter equals end + step.
- If start is already beyond end for the given Step, the body runs **zero** times.

Program 15 - For...Next - table and countdown

```
Dim i As Integer, s As String
For i = 1 To 5
    Print "5 x " & i & " = " & 5 * i
Next i
For i = 5 To 1 Step -1
    s = s & i & " "
Next i
Print "Countdown: " & s
```

```
OUTPUT
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
Countdown: 5 4 3 2 1
```

4.6 For Each ... Next

Definition: Visits **every element** of an array or a collection (such as `Me.Controls`) one by one, without using an index or counting.

Syntax

```
For Each element In group
    statements
Next [element]
```

Rules

- The loop variable must be a **Variant** when looping through an array (Variant or Object for a collection).
- It receives a **copy** of each element, so changing it does not change the array.
- The loop runs once for every element and does nothing for an empty group.

Program 16 - For Each - names in an array

```
Dim names As Variant, nm As Variant
names = Array("Amit", "Bina", "Chitra")
For Each nm In names
    Print nm
Next nm
```

```
OUTPUT
Amit
Bina
Chitra
```

4.7 Nested Loops

Definition: A loop placed **inside** another loop. The outer loop controls the rows or main repetition; the inner loop runs **completely** for every single pass of the outer loop.

Syntax

```
For outer = 1 To m
    For inner = 1 To n
        statements              'runs m x n times
    Next inner
Next outer
```

How it works

- When the outer counter is 1, the inner loop runs from 1 to n. When the outer counter is 2, the inner loop again runs from 1 to n, and so on until the outer loop ends.
- Total passes of the inner body = outer passes x inner passes (3 x 4 = 12).
- The inner loop must be closed (`Next inner`) **before** the outer loop.
- Any combination of loop types can be nested (For inside Do, Do inside For).

Program 17 - nested For loops - star pattern

```
Dim i As Integer, j As Integer, s As String
For i = 1 To 4                  'outer loop: rows
    s = ""
    For j = 1 To i              'inner loop: stars in this row
        s = s & "* "
    Next j
    Print s
Next i
```

```
OUTPUT
*
* *
* * *
* * * *
```

4.8 Exit Statements (Exit For, Exit Do, Exit Sub)

Definition: An **Exit** statement leaves a loop or procedure **immediately**, before its normal end, usually when an `If` test shows the work is finished.

Statement	Used in	Effect
`Exit For`	For...Next, For Each...Next	Leaves the **innermost** For loop; execution continues after `Next`
`Exit Do`	All Do loops	Leaves the innermost Do loop; execution continues after `Loop`
`Exit Sub`	Sub procedures	Leaves the **whole procedure** (any loop inside also ends); `Exit Function` does the same in a Function

Program 18 - Exit For - first multiple of 7

```
Dim i As Integer
For i = 50 To 100
    If i Mod 7 = 0 Then Exit For
Next i
Print "First multiple of 7 = " & i
```

```
OUTPUT
First multiple of 7 = 56
```

Program 19 - Exit Do - controlled exit

```
Dim i As Integer
i = 0
Do
    i = i + 1
    If i > 3 Then Exit Do
    Print "Pass " & i
Loop
```

```
OUTPUT
Pass 1
Pass 2
Pass 3
```

Program 20 - Exit Sub - stop the procedure early

```
Private Sub Command1_Click()
    If Text1.Text = "" Then
        MsgBox "Enter a value"
        Exit Sub                'remaining statements are skipped
    End If
    MsgBox "Hello " & Text1.Text
End Sub
```

```
OUTPUT
Text1 empty:      only the message "Enter a value" appears
Text1 = "Amit":   Hello Amit
```

REMEMBER: `Exit For` and `Exit Do` leave only the **innermost** loop; to leave nested loops completely use `Exit Sub` or a flag variable.

4.9 Comparison of Loops

Loop	Condition tested	Minimum runs	Repeats while	Best used when
`Do While ... Loop`	Before body	0	Condition True	Count unknown; may need zero passes
`Do ... Loop While`	After body	1	Condition True	Body must run once (e.g. menu, input)
`Do Until ... Loop`	Before body	0	Condition False	Run until a target is reached
`Do ... Loop Until`	After body	1	Condition False	Repeat until valid input entered
`For ... Next`	Before each pass	0	Counter within limit	Count known in advance
`For Each ... Next`	Elements remain	0 (empty group)	Elements left	Process every array element / collection item

4.10 Infinite Loops and Incorrect Termination Conditions

Definition: An **infinite loop** is a loop whose terminating condition is never reached, so it repeats forever and the program appears to hang. An **incorrect termination condition** makes a loop stop too early, never run, or run once too often.

Faulty code	Problem	Fix
`i = 1: Do While i <= 5 : Print i : Loop`	No update, i stays 1: **infinite loop**	Add `i = i + 1`
`i = 1: Do Until i = 10 : i = i + 2 : Loop`	i takes 1,3,5,7,9,11... never equals 10: **infinite**	Use `Do Until i >= 10`
`Do While True ... Loop`	Condition always True, no exit	Add `If ... Then Exit Do`
`For i = 1 To 5 : i = i - 1 : Next`	Changing the counter inside the loop keeps it at 0: infinite	Never modify the For counter
`For i = 10 To 1`	Start > end with positive Step: body runs **0 times**	`For i = 10 To 1 Step -1`
`For i = 0 To UBound(a) + 1`	Off-by-one: last pass is out of range (Error 9)	`For i = 0 To UBound(a)`

TIP: In the VB IDE press **Ctrl+Break** to stop a runaway loop.

5. Arrays

An **array** is a group of variables of the **same data type** that share one name; each element is reached by a number called its **index** (subscript). **Purpose:** store and process many related values (marks of 50 students) without declaring 50 separate variables.

- **Advantages:** one name for many values; easy processing with loops; compact code; fast access to any element by index.

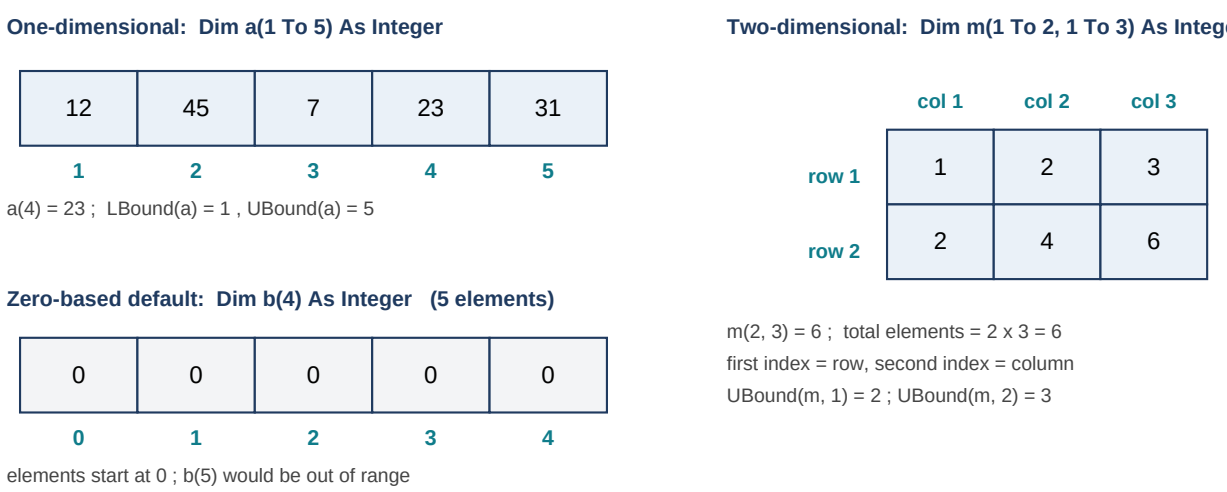


Fig 5: One-dimensional and two-dimensional arrays with indexes

5.1 Declaration, indexing and bounds

Syntax - declaring arrays

```
Dim a(4) As Integer              'indexes 0..4  -> 5 elements (default lower bound is 0)
Dim marks(1 To 5) As Integer     'indexes 1..5  -> 5 elements
Dim b(2 To 8) As Single          'LBound = 2, UBound = 8 -> 7 elements
a(0) = 10: a(4) = 50             'initialise by assigning to elements
```

- **Bounds:** lower bound = first valid index, upper bound = last valid index. Elements = upper - lower + 1. `Dim a(4)` has **5** elements, not 4.
- `LBound(arr)` returns the lowest index, `UBound(arr)` the highest. For multidimensional arrays give the dimension: `UBound(m, 2)`. Using them in loops keeps code correct if the size changes.
- Numeric elements start as 0, strings as "". Placing `Option Base 1` in Declarations makes the default lower bound 1.
- Array elements are accessed as `a(i)` - the index can be a constant, variable or expression.

5.2 Fixed-size and dynamic arrays

	Fixed-size array	Dynamic array
Declaration	`Dim a(1 To 10) As Integer`	`Dim a() As Integer` then `ReDim a(1 To n)`
Size	Constant, fixed at design time	Decided / changed at run time using a variable
Resize	Not possible	`ReDim` (erases data) or `ReDim Preserve` (keeps data)
Release memory	No	Yes - `Erase a`
Use when	Size known in advance	Size depends on user input or data

- `ReDim Preserve` can change only the **last** dimension of a multidimensional array.
- The size of a fixed array must be a **constant** or literal: `Dim a(n)` with a variable n is a compile error.

Program 21 - dynamic array (ReDim Preserve)

```
Dim a() As Integer
ReDim a(1 To 3)
a(1) = 10: a(2) = 20: a(3) = 30
ReDim Preserve a(1 To 5)             'grow, keep old values
a(4) = 40: a(5) = 50
Print "UBound = " & UBound(a) & ", a(2) = " & a(2)
ReDim a(1 To 5)                      'without Preserve all values are cleared
Print "After ReDim a(2) = " & a(2)
```

```
OUTPUT
UBound = 5, a(2) = 20
After ReDim a(2) = 0
```

5.3 One-dimensional array: enter, display, process, search

Program 22 - one-dimensional array (enter, display, process, search)

```
Private Sub Form_Click()
    Dim a(1 To 5) As Integer, i As Integer, s As String
    Dim sum As Integer, big As Integer, small As Integer, key As Integer, found As Boolean
    For i = 1 To 5                                  'ENTER (assume user types 12 45 7 23 31)
        a(i) = InputBox("Enter number " & i)
    Next i
    For i = LBound(a) To UBound(a)                  'DISPLAY
        s = s & a(i) & " "
    Next i
    Print "Numbers: " & s
    big = a(1): small = a(1)                        'PROCESS: sum, max, min
    For i = 1 To 5
        sum = sum + a(i)
        If a(i) > big Then big = a(i)
        If a(i) < small Then small = a(i)
    Next i
    Print "Sum = " & sum & "  Average = " & sum / 5
    Print "Largest = " & big & "  Smallest = " & small
    key = 23                                        'SEARCH (linear search)
    For i = LBound(a) To UBound(a)
        If a(i) = key Then found = True: Exit For
    Next i
    If found Then Print key & " found at index " & i Else Print key & " not found"
End Sub
```

```
OUTPUT
Numbers: 12 45 7 23 31
Sum = 118  Average = 23.6
Largest = 45  Smallest = 7
23 found at index 4
```

5.4 Multidimensional array

Declared with several subscripts: **`Dim m(1 To 2, 1 To 3)`** is a table of 2 rows x 3 columns (6 elements). Typical processing uses **nested For loops** - outer for rows, inner for columns. VB allows up to 60 dimensions.

Program 23 - two-dimensional array

```
Dim m(1 To 2, 1 To 3) As Integer, i As Integer, j As Integer
Dim s As String, total As Integer
For i = 1 To 2
    s = ""
    For j = 1 To 3
        m(i, j) = i * j
        s = s & m(i, j) & " "
        total = total + m(i, j)
    Next j
    Print s
Next i
Print "Total = " & total
```

```
OUTPUT
1 2 3
2 4 6
Total = 18
```

5.5 Common errors

Error	Cause	Message
`Dim a(4): a(5) = 1` or **`a(-1)`**	Invalid index - outside LBound..UBound	Run-time error 9: Subscript out of range
`Dim a(n) As Integer` (n variable)	Fixed array needs a constant size	Compile error: Constant expression required
`Dim a(5)` then **`ReDim a(10)`**	ReDim works only on dynamic arrays	Compile error: Array already dimensioned
`Dim a() As Integer: a(1) = 5`	Dynamic array used before ReDim	Run-time error 9: Subscript out of range
`ReDim Preserve m(1 To 5, 1 To 3)`	Preserve changed a non-last dimension	Run-time error 9: Subscript out of range
`a(1) = "abc"` (Integer array)	Wrong data type for the element	Run-time error 13: Type mismatch
`Dim a(5)` assumed to hold 5 items	It holds 6 (0..5) - classic off-by-one	Logic error - use **`1 To 5`**

6. User Defined Procedure, Functions

A **procedure** is a named block of code written once and **called** whenever needed. **Purpose:** avoid repeated code, break a big problem into small parts, ease testing and reuse. Besides built-in event procedures (`Command1_Click`), the programmer writes **general (user-defined) procedures** of two kinds: **Sub** (does a task, returns nothing) and **Function** (does a task and **returns one value**).

6.1 Sub procedure

Definition: A **Sub procedure** is a named block of code that performs a task but **does not return a value**. It begins with `Sub`, ends with `End Sub`, and runs when it is called by name (or with `Call`).

Syntax - Sub procedure and calling

```
[Public|Private] Sub SubName([parameter list])        'declaration (header) ...
    statements                                        '... plus body = definition
End Sub

SubName arg1, arg2          'calling - no parentheses
Call SubName(arg1, arg2)    'calling with Call - parentheses required
```

- **Parameters** are the variables listed in the procedure header; **arguments** are the actual values supplied at the call.
- **Declaration/definition:** the header line declares name and parameters; header + body + `End Sub` define the procedure. It is written once in the code window (Tools > Add Procedure creates the skeleton).
- `Public` procedures (in a .bas module) can be called from any form; `Private` ones only within their own module.

Program 24 - Sub procedures

```
Private Sub ShowLine()                      'without arguments
    Print "--------------------"
End Sub
Private Sub Greet(ByVal nm As String)       'with an argument
    Print "Hello, " & nm
End Sub
Private Sub AddNum(a As Integer, b As Integer)   'with two arguments
    Print a & " + " & b & " = " & (a + b)
End Sub

Private Sub Form_Click()
    ShowLine
    Greet "Amit"
    Call Greet("Bina")
    AddNum 7, 5
    ShowLine
End Sub
```

```
OUTPUT
--------------------
Hello, Amit
Hello, Bina
7 + 5 = 12
--------------------
```

6.2 Function procedure and returning a value

Definition: A **Function procedure** is a named block of code that performs a task and **returns a single value** to the place where it was called. It begins with `Function`, ends with `End Function`, and is used inside expressions, assignments or `Print` statements.

Syntax - Function procedure

```
[Public|Private] Function FuncName([parameters]) [As ReturnType]
    statements
    FuncName = value            'return value: assign to the function's OWN NAME
End Function

result = FuncName(arg1, arg2)   'call inside an expression, assignment or Print
```

- A Function returns its value by **assigning it to the function name** (VB has no `return` keyword). If no value is assigned, the default for the type (0, "" ...) is returned.
- `Exit Function` ends the function early. The returned value can be stored, printed or used in another expression.
- Parentheses around arguments are needed when the returned value is used.

Program 25 - Function procedures

```
Private Function GetTax() As Single                 'without arguments
    GetTax = 0.18
End Function

Private Function Square(n As Integer) As Long       'with an argument
    Square = n * n
End Function

Private Function Fact(ByVal n As Integer) As Long   'factorial
    Dim i As Integer, f As Long
    f = 1
    For i = 2 To n
        f = f * i
    Next i
    Fact = f
End Function

Private Function Max(a As Integer, b As Integer) As Integer
    If a > b Then Max = a Else Max = b
End Function

Private Sub Form_Click()
    Print "Tax rate = " & GetTax()
    Print "Square(5) = " & Square(5)
    Print "5! = " & Fact(5)
    Print "Max(8, 13) = " & Max(8, 13)
End Sub
```

```
OUTPUT
Tax rate = 0.18
Square(5) = 25
5! = 120
Max(8, 13) = 13
```

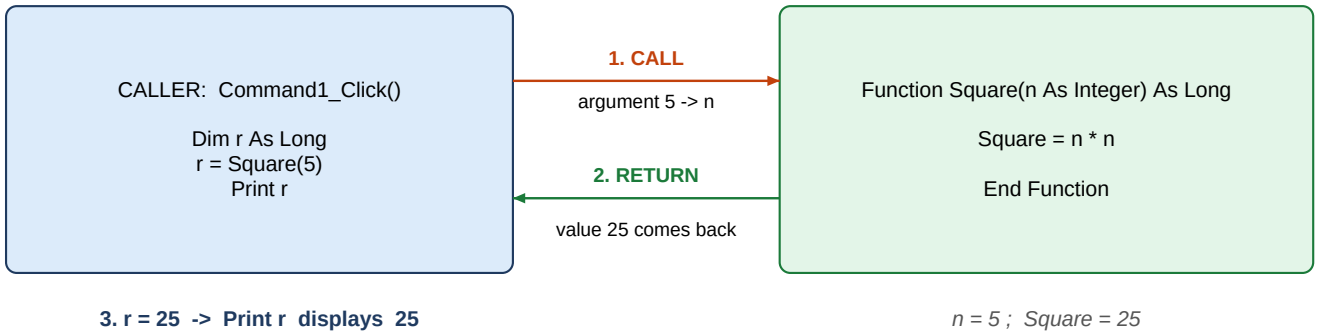


Fig 6: Function call and return flow

6.3 Passing arguments: ByVal and ByRef

	ByRef (default)	ByVal
What is passed	Address (reference) of the original variable	A **copy** of the value
Effect of changes in procedure	Changes the **caller's** variable	Caller's variable stays unchanged
Use when	Procedure must return/modify several values (swap)	Value is only to be read (safe)
Keyword needed?	No - used when nothing is written	`ByVal` must be written

Program 26 - ByVal and ByRef

```
Private Sub SwapR(ByRef a As Integer, ByRef b As Integer)
    Dim t As Integer
    t = a: a = b: b = t
End Sub
Private Sub SwapV(ByVal a As Integer, ByVal b As Integer)
    Dim t As Integer
    t = a: a = b: b = t
End Sub
Private Sub Form_Click()
    Dim x As Integer, y As Integer
    x = 5: y = 9
    SwapV x, y
    Print "After ByVal : x = " & x & ", y = " & y
    SwapR x, y
    Print "After ByRef : x = " & x & ", y = " & y
End Sub
```

```
OUTPUT
After ByVal : x = 5, y = 9
After ByRef : x = 9, y = 5
```

REMEMBER: `Optional ByVal n As Integer = 1` makes an argument optional with a default. Writing `Greet (x)` (extra parentheses, no Call) passes x by value even if the parameter is ByRef.

6.4 Sub vs Function

Point	Sub procedure	Function procedure
Returns a value	No	Yes - exactly one value (type given by `As`)
How value is returned	Not applicable (can change ByRef arguments)	Assign to function name
Called as	`Greet "Amit"` or `Call Greet("Amit")`	`x = Square(4)` or `Print Square(4)`
Used in an expression	No	Yes
Early exit	`Exit Sub`	`Exit Function`
Typical use	Perform an action (print, clear, update)	Calculate and give back a result

7. VB Functions

Built-in functions are ready-made routines that take **arguments** (inputs) and **return a value**. They are grouped into four categories below.

7.1 Date functions

Definition: Date functions **read, calculate and compare** dates and times, including the current system date and time, and return a date or a number.

Function	Syntax	Returns
`Date`	`Date`	Current system date
`Time`	`Time`	Current system time
`Now`	`Now`	Current system date and time
`Day / Month / Year`	`Day(date)`	Day of month (1-31) / month (1-12) / year of the date
`Weekday`	`Weekday(date)`	Day of week as number: Sunday = 1 ... Saturday = 7
`DateAdd`	`DateAdd(interval, number, date)`	Date after adding *number* intervals (negative = subtract)
`DateDiff`	`DateDiff(interval, date1, date2)`	Number of intervals from date1 to date2 (negative if date2 is earlier)

Interval codes (used in DateAdd and DateDiff)

Interval	Meaning	Example
`yyyy`	Year	`DateAdd("yyyy", 1, d)` adds 1 year
`q`	Quarter	`DateAdd("q", 1, d)` adds 3 months
`m`	Month	`DateAdd("m", 2, d)` adds 2 months
`y`	Day of year	`DateAdd("y", 5, d)` adds 5 days
`d`	Day	`DateAdd("d", 10, d)` adds 10 days
`w`	Weekday	`DateAdd("w", 3, d)` adds 3 days
`ww`	Week	`DateAdd("ww", 2, d)` adds 14 days
`h`	Hour	`DateAdd("h", 5, Now)` adds 5 hours
`n`	Minute	`DateAdd("n", 30, Now)` adds 30 minutes
`s`	Second	`DateAdd("s", 45, Now)` adds 45 seconds

Program 27 - Date functions

```
Dim d As Date
d = #8/15/2026#                                  'date literals use m/d/yyyy
Print Day(d) & "  " & Month(d) & "  " & Year(d)
Print "Weekday = " & Weekday(d)
Print Format(DateAdd("d", 10, d), "dd-mmm-yyyy")
Print Format(DateAdd("m", -2, d), "dd-mmm-yyyy")
Print Format(DateAdd("m", 1, #1/31/2026#), "dd-mmm-yyyy")
Print "Days from 1 Jan to d = " & DateDiff("d", #1/1/2026#, d)
Print "Months = " & DateDiff("m", #1/1/2026#, d)
Print "Reverse = " & DateDiff("d", d, #1/1/2026#)
Print "Year boundary = " & DateDiff("yyyy", #12/31/2025#, #1/1/2026#)
If d < #12/31/2026# Then Print "d is earlier than 31 Dec 2026"
```

```
OUTPUT
15  8  2026
Weekday = 7
25-Aug-2026
15-Jun-2026
28-Feb-2026
Days from 1 Jan to d = 226
Months = 7
Reverse = -226
Year boundary = 1
d is earlier than 31 Dec 2026
```

- **Format()** is used here only to fix the display style; the plain `Print d` shows the date according to the computer's Regional Settings.
- **Output depends on the system date/time:** if the clock shows 12 Oct 2026, 3:45 PM then `Date` gives 12-10-2026 (format varies), `Time` gives 3:45:30 PM, `Now` gives both, `Day(Date)` = 12, `Month(Date)` = 10, `Year(Date)` = 2026, `Weekday(Date)` = 2 (Monday).

- DateAdd clamps to the last valid day (31 Jan + 1 month = 28 Feb). DateDiff counts **boundaries crossed**, not full intervals: `DateDiff("yyyy", #12/31/2025#, #1/1/2026#)` = 1 although only one day has passed.
- Dates compare with `< > =` like numbers; e.g. age in years: `DateDiff("yyyy", dob, Date)` (approximate).

7.2 Mathematical functions

Definition: Mathematical functions perform **numeric calculations** such as absolute value, square root, rounding and random numbers, and return a number.

Function	Syntax	Returns	Examples
`Abs`	`Abs(n)`	Absolute (non-negative) value	`Abs(-7.5)` = 7.5
`Sqr`	`Sqr(n)`	Square root (n must be >= 0)	`Sqr(144)` = 12 ; `Sqr(2)` = 1.4142135623731
`Sgn`	`Sgn(n)`	Sign: -1, 0 or 1	`Sgn(-9)` = -1 ; `Sgn(0)` = 0 ; `Sgn(4)` = 1
`Int`	`Int(n)`	Largest integer <= n (rounds **down**)	`Int(4.7)` = 4 ; `Int(-4.7)` = -5
`Fix`	`Fix(n)`	Integer part (**truncates** decimals)	`Fix(4.7)` = 4 ; `Fix(-4.7)` = -4
`Round`	`Round(n [, places])`	Rounds to given decimal places; .5 goes to the **even** digit (banker's rounding)	`Round(3.14159, 2)` = 3.14 ; `Round(2.5)` = 2 ; `Round(3.5)` = 4
`Rnd`	`Rnd[(n)]`	Random Single, 0 <= value < 1	`Randomize` first, else the same sequence repeats each run

x	Int(x)	Fix(x)	Reason
4.7	4	4	Same for positive numbers
-4.7	-5	-4	Int goes to the lower integer, Fix just drops the decimals
6	6	6	Whole numbers are unchanged

Program 28 - Mathematical functions

```
Print Abs(-7.5), Sqr(144), Sgn(-9)
Print Int(-4.7), Fix(-4.7), Round(3.14159, 2)
Print Round(2.5), Round(3.5), Int(2.5 + 0.5)          'Int(x + 0.5) = ordinary round-half-up
Randomize
Print "Dice = " & Int(Rnd * 6) + 1                    'random whole number 1 to 6
'general formula: Int((high - low + 1) * Rnd + low)
```

```
OUTPUT
 7.5           12           -1
-5            -4            3.14
 2             4            3
Dice = (a random value from 1 to 6)
```

REMEMBER: **Exam practice:** `Int(-7.2) + Fix(-7.2)` = -8 + -7 = **-15**; `Sqr(Abs(-16))` = **4**; `Round(4.5) + Round(5.5)` = 4 + 6 = **10**; `Sqr(-4)` gives run-time error 5 (Invalid procedure call).

7.3 String functions

Definition: String functions **measure, extract, search, change or replace** text (a sequence of characters) and return the result.

Take **s = "Visual Basic"**. Positions: V1 i2 s3 u4 a5 l6 (space)7 B8 a9 s10 i11 c12.

Function	Syntax	Returns	Example = result
`Len`	`Len(string)`	Number of characters (spaces count)	`Len(s)` = **12**
`Left`	`Left(string, n)`	First n characters	`Left(s, 6)` = **Visual**
`Right`	`Right(string, n)`	Last n characters	`Right(s, 5)` = **Basic**
`Mid`	`Mid(string, start [, n])`	n characters from position start (1 = first); to the end if n is omitted	`Mid(s, 8, 3)` = **Bas** `Mid(s, 8)` = **Basic**
`Trim`	`Trim(string)`	Removes leading and trailing spaces (LTrim: left only, RTrim: right only)	`Trim(" hi ")` = **hi (Len = 2)**
`LCase / UCase`	`UCase(string)`	All lower-case / all upper-case	`UCase(s)` = **VISUAL BASIC** `LCase(s)` = **visual basic**
`InStr`	`InStr([start,] string1, string2 [, compare])`	Position of string2 inside string1; **0** if not found. Case-sensitive unless vbTextCompare is given	`InStr(s, "Basic")` = **8** `InStr(s, "xyz")` = **0** `InStr("Visual", "VIS")` = **0** `InStr(1, "Visual", "VIS", vbTextCompare)` = **1**

Function	Syntax	Returns	Example = result
`Replace`	`Replace(string, find, replacewith [, start [, count]])`	String with every occurrence of find replaced	`Replace(s, "Basic", "Script")` = **Visual Script** `Replace("a-b-c", "-", "/", 1, 1)` = **a/b-c**

Program 29 - String functions (email split)

```
Dim e As String, p As Integer
e = "amit@oavs.org"
p = InStr(e, "@")                                   'position of @ = 5
Print "User   : " & Left(e, p - 1)
Print "Domain : " & Mid(e, p + 1)
Print "Upper  : " & UCase(Left(e, 1)) & Mid(e, 2, p - 2)
```

```
OUTPUT
User   : amit
Domain : oavs.org
Upper  : Amit
```

REMEMBER: `Left(s, 20)` on a 12-character string returns the whole string (no error); `Mid(s, 20)` returns "" (empty). Position numbers start at **1**, not 0.

7.4 Conversion functions

Definition: Conversion functions change a value from **one data type to another** (for example text to number, or number to text) so that it can be used correctly in calculations or display. They are needed because a text box always holds **text**, so **Text1.Text + Text2.Text** with "10" and "20" gives "1020" (joined), not 30.

Function	Converts to	Notes and examples
`CInt(x)`	Integer	Rounds (.5 to even): `CInt(7.5)` = 8, `CInt(6.5)` = 6; Error 6 if outside -32,768 to 32,767
`CLng(x)`	Long	Same rounding; larger range: `CLng("40000")` = 40000
`CSng(x)`	Single	`CSng("3.14159265")` = 3.141593 (Single has about 7 digits of precision)
`CDbl(x)`	Double	`CDbl("2.5") * 2` = 5
`CStr(x)`	String	`CStr(123) & "4"` = "1234" ; `CStr(True)` = "True"; no leading space
`CDate(x)`	Date	`Month(CDate("15-Aug-2026"))` = 8; invalid text gives Error 13
`Val(string)`	Number (Double)	Reads digits from the **start** of the text and stops at the first non-numeric character; returns 0 if none; **never gives an error**
`Str(number)`	String	Adds a **leading space** for the sign of positive numbers: `Str(25)` = " 25", `Str(-25)` = "-25"

Program 30 - Conversion functions

```
Print CInt(7.5), CInt(6.5), CLng("40000")
Print CSng("3.14159265"), CDbl("2.5") * 2, CStr(123) & "4"
Print Val("123abc"), Val("abc"), Val("12.5kg"), Val(" 45 ")
Print "[" & Str(25) & "]", "[" & Str(-25) & "]", "[" & CStr(25) & "]"
Print Month(CDate("15-Aug-2026"))
'Text1.Text = "10" and Text2.Text = "20"
Print Text1.Text + Text2.Text, Val(Text1.Text) + Val(Text2.Text)
```

```
OUTPUT
 8             6             40000
 3.141593      5             1234
 123           0             12.5          45
[ 25]         [-25]         [25]
 8
1020          30
```

Compare	Difference
`CInt` / `Int` / `Fix`	For 2.7: CInt = 3 (rounds), Int = 2 (floor), Fix = 2 (truncate); for -2.7: CInt = -3, Int = -3, Fix = -2
`Val` vs `CInt`	`Val("12abc")` = 12 with no error; `CInt("12abc")` = Type mismatch
`Str` vs `CStr`	Str keeps a sign-space for positive numbers; CStr does not

Common conversion errors

Statement	Error
`CInt("abc")` , `CInt("")` , `CInt("12abc")`	Run-time error 13: Type mismatch
`CInt(40000)`	Run-time error 6: Overflow (use CLng)
`CDate("31/02/2026")` , `CDate("abc")`	Run-time error 13: Type mismatch (invalid date)
Using Text1.Text + Text2.Text for addition	No error, but gives joined text "1020" - convert with Val or CDbl first

TIP: Check input before converting: `If IsNumeric(Text1.Text) Then n = CInt(Text1.Text)` and `IsDate()` for dates, to avoid Type mismatch errors.

Quick Revision

Topic	Key points to remember
Variables / constants	Dim x As Type; names start with a letter, no spaces, no reserved words; default 0 / "" / False; Const cannot be changed; `Dim a, b As Integer` makes a Variant
Data types	Integer 2 B, Long 4 B, Single 4 B, Double 8 B, Currency 8 B, Boolean 2 B, Date 8 B, Variant 16 B; UDT = Type...End Type, members via dot
Scope	Local = Dim in procedure; Modular = Dim/Private in declarations; Global = Public in .bas; Static = local scope, whole-run lifetime
Control flow	If / ElseIf / Else / End If; Select Case (first match, Case Else); IIf evaluates BOTH parts and returns a value
Loops	Do While / Until test first; Loop While / Until test last (runs once); For known count; For Each over arrays/collections; Exit For / Do / Sub
Arrays	Dim a(n) has n+1 elements from 0; LBound/UBound; ReDim [Preserve] for dynamic arrays; Error 9 = bad subscript
Procedures	Sub does a task; Function returns value via its own name; ByRef (default) changes caller, ByVal copies; parameter = in header, argument = in call
Date	Date, Time, Now, Day, Month, Year, Weekday; DateAdd(interval, n, d); DateDiff(interval, d1, d2) counts boundaries
Math	Abs, Sqr, Sgn; Int floors, Fix truncates (differ for negatives); Round is banker's; Randomize then Int(Rnd*N)+1
String	Len, Left, Right, Mid (1-based), Trim, LCase, UCase, InStr (0 = not found), Replace
Conversion	CInt/CLng/CSng/CDbl/CStr/CDate; CInt rounds, Overflow if too big; Val never errors; Str adds leading space